

Term (术语)	Definition (定义)	Example (示例)
	要程度	
Context Vector (上下文向量)	用注意力权重对编码器隐藏状态做加权求和得到的向量。通常，它结合了句子的信息。	$c = 0.7 \cdot h_1 + 0.1 \cdot h_2 + 0.1 \cdot h_3 + \dots$

Down Product Attention (点积主要注意)	向量点积面积 (内积) 计算注意力分数的方法, 衡量每个词和输入序列的相似度	score = query * T^key
Attention Output (注意力输出)	将解码器输出和上下文向量拼接后经过注意力函数, 再通过 softmax 预测输出	output = [id; dec; state; context] → softmax → word

Transformer Preview (Transformer 预览)

Term (术语)	Definition (定义)	Example (示例)
Transformer (变换器, 2017)	完全依靠注意力方法架构, 彻底不需要 RNN, 可以完全替代并处理序列	2017年论文提出 → 催生 GPT, BERT 等神经网络模型
Self-Attention (自注意力)	序列中每个位置与序列中所有其他位置 (包括自己), 都向序列中 Encoder → Decoder	"I" 关注前面的 "cat" 来理解整句的意思, 无法理解前句
Positional Encoding (位置编码)	Encoder/Transformer 没有循环结构, 需要添加位置位置编码来区分序列中每个位置, 否则模型无法区分前后	给每个位置一个无法训练的向量, 如位置编码不随输入而变
Transformer Architecture (架构)	编码器输入 Self-Attn + FFN → 解码器输入 Self-Attn + Cross-Attn + FFN, 含 dropout	解码器: Self-Attn → FFN; 训练器: Self-Attn → Cross-Attn → FFN

Comparisons (对比)

Unit-LSTM vs Bi-LSTM (单向 vs 双向)	
Dimension (维度)	Unit-LSTM (单向) Bi-LSTM (双向)
Direction (方向)	过去 → 未来 (→) 过去 ← 未来 (←)
Output dim (输出维度)	hidden size 2 * hidden size
	"terribly" 需要看到后面才断正正确义 LSTM (128) + LSTM (128) = 256d

Context (上下文)	只有左侧上下文	左右两侧完整上下文	Bi-LSTM 理解更全面
Generation (生成)	✅ 可以生成文本	❌ 不能生成 (需要未来信息才能运行)	自回归生成需要从左到右, 反向 LSTM 需要整个序列已存在
Use case (场景)	文本生成, 语言模型	分类, NER, 问答	分类用 Bi-LSTM; 生成用 Uni

Weights (权重)	一般权重	两组独立权重 (不共享)	W _{forward} & W _{backward}
Seq2Seq vs Seq2Seq + Attention			
Dimension (维度)	Seq2Seq (原始)	Seq2Seq + Attention	Example (示例)
Info path (信息路径)	仅通过单一固定编码向量 (瓶颈)	无固定编码向量 每个输入都有编码路径	瓶颈 vs 直通通
Long sequences (长序列)	容易丢失开头 (X) 需要重新初始化	无瓶颈, 注意力可随时间变化	100句子的文档生成90句子的摘要
Interpretability (可解释性)	无信息, 注意力不可可视化	注意力权重可视化 = 好问题	能够知道每个词何时关注了哪些内容
Gradient (梯度)	只能通过 RNN 传播	注意力路径提供快捷路径	梯度流——每个词都有梯度
Processing (处理方案)	RNN 顺序处理	仍然是 RNN 顺序处理 (不能并行!)	都是基于 RNN 基, 都能并行
Cost per step (每步开销)	O(1)——只用编码向量	O(n)——每步都有看输入	训练时计算量更大 推理时更小
Training vs Inference (训练 vs 推理)			
Phase (阶段)	Input to decoder (输入到解码器)	Issue (问题)	Example (示例)
Training (训练)	每个词自己的子目标	快速收敛或失败	真实 "I" 预测 "love"
Inference (Auto-Inference)	模型自己上一个预测词	训练会收敛或失败	预测 "like" (输入: 后话)

regressive)	Gap = Exposure Bias	训练/推理输入不一致 误差像滚雪球越来越大	积偏 训练从没见过自己的错误 → 推理时跑题
-------------	---------------------	--------------------------	------------------------------

RNN Attention vs Self-Attention (编解码注意力 vs 自注意力)

Dimension (维度)	RNN Attention (本讲重点)	Self-Attention (Transformer)	Example (示例)
Who attends (谁关注谁)	Decoder → Encoder (解码器看编码器)	每个词 → 所有词 (包括自己)	翻译对齐 vs 自身顺序理解

Depends on RNN	✔ 需要 RNN 作为骨架	✘ 完全不需要 RNN	Transformer 完全去掉 RNN
Parallel (并行)	✘ RNN 必须顺序处理	✔ 完全并行	Transformer 训练快得多

Long-range path (长距离)	需要多少 RNN 传播	$O(1)$ 路径长度, 任意两位置直接连接	远处的词也能直接关注
Position info (位)	隐式 (RNN 处理顺序)	显式 (必须加 Positional)	没有循环一必须额

From → To	Problem Solved (解决了什么)	Cost (代价)
-----------	------------------------	-----------

单向LSTM → Bi-LSTM	方向盲区：只看左边 → 看两边完整上方向	不能生成文本：参数翻倍(2n)
Bi-LSTM → Seq2Seq	长度问题：无法处理输入输出不等长 → 编码器+解码器两阶段	信息瓶颈(整个句子 → 一个向量)
Seq2Seq → + Attention	信息瓶颈：一个向量装不下 → 每一步动态加权重所有位置	仍需 RNN 顺序处理：每步计算 $O(n)$
RNN + Attention → Transformer	顺序处理：RNN 不能并行 → 自注意力完全并行	$O(n^2)$ 内存开销：需要位置编码

Attention Mechanism 4 Steps (注意力机制4步)			
Step (步)	Formula (公式)	Description (说)	Example (示例)

1. Score (打分)	$s_i = \text{enc_state}^T \cdot \text{enc_state}_i$	解码器状态与每个编码器状态的点积相似度	4个编码器状态 → 4个分数
---------------	---	---------------------	----------------

2. Normalize (归一化)	$\alpha_i = \frac{\exp(c_i)}{\sum_j \exp(c_j)}$	用 softmax 转为概率分布 (和=1)	$[3,1,1] \rightarrow [0.7,0.1,0.1,0.1]$
3. Context (上下文)	$c = \sum_i \alpha_i \cdot \text{enc_states}_i$	用权重对编码器状态做加权和	主要包含高权重位置的信息
4. Output (输出)	$\text{output} = f(\text{enc_states}_q, c)$	拼接解码器状态和上下文向量 → 全连接	拼接后经过全连接层预测目标词

		→ softmax	
--	--	-----------	--

Bi-LSTM Output (Bi-LSTM 输出)

Formula (公式)	Description (说明)	Example (示例)
--------------	------------------	--------------

$h_{ij} = \hat{h}_{ij}^L$	正向和反向隐状态拼成完整表示	每个位置都有左右两侧上下文
$\dim(h_{ij}) = 2 \times \text{Model}_{\text{hidden}}$	输出维度是单向的两倍	Bi-LSTM(128) 输出 256 维
Conditional LM (条件语言模型公式)		
Formula (公式)	Description (说明)	Example (示例)
$P(y_1, \dots, y_T) = \prod_{t=1}^T P(y_t x_1, \dots, x_{t-1})$	普通语言模型：只依赖生成前上下文	无条件自由生成文本
$P(y_1, \dots, y_T x) = \prod_{t=1}^T P(y_t x_1, \dots, x_{t-1}, x)$	条件语言模型：条件由输入序列 x 提供	翻译时每个词同时依赖已翻译的词和源句
NMT Training Loss (NMT 训练损失)		
Formula (公式)	Description (说明)	Example (示例)

$J_t = -\log P(y_t x_1, \dots, x_{t-1}, y_t)$	第 t 步的交叉熵损失 (单个目标的负对数概率)	目标词 "hit" 的预测概率越高 $\rightarrow$ 1, 越小 $\rightarrow$ 0
$J = -\frac{1}{T} \sum_{t=1}^T J_t$	整个序列的平均损失, 端到端反向传播优化此目标	所有时间步损失的均值 $\rightarrow$ 一个标量 $\rightarrow$ 梯度反传

Score	$e = [0.1, 0.05, 0.02, 0.83]$	$\text{dot}(\text{dec}_h, \text{enc}_h)$, "entarté" 最相似
Softmax	$\alpha \approx [0.10, 0.09, 0.09, 0.72]$	"entarté" 获得最高注意力权重 0.72
Context	$c = 0.10 \cdot h_1 + 0.09 \cdot h_2 + 0.09 \cdot h_3 + 0.72 \cdot h_4$	上下文向量主要包含 "entarté" 信息

Output	[dec_h; c] → FC → softmax → 'pie'	拼接后预测出正确的目标词
--------	-----------------------------------	--------------

四. Practical / Lab (实践结论)


Key Distinctions (关键区别)

Distinction (区别)	Detail (详情)	Example (示例)
Bi-LSTM 输出维度 = $2 \times \text{hidden_dim}$	BiDirectional(LSTM(128)) → 输出 256 维, 值为正负	hidden=128 → 正向 128→后向 128=256

model_size	128+反向128维隐层	128+反向128+256
return_sequences=True vs False	True=返回所有时间步输出(Attention/Seq2Seq需要); False(默认)=只返回最后一步(分类用)	True: shape(10,256); False: shape(256).
正反向 LSTM 有独立权重	两个方向不共享参数, 各自独立训练不同的权重矩阵	W_forward ≠ W_backward

Attention 权重是动态计算的	不是训练出来的固定参数; 每个新输入/时间步都要通过点积+softmax重新计算	换一个词/字 → 注意力权重完全不同
Seq2Seq 短句好 长句差	编码向容量固定 → 句子越长丢失信息越多(<20词还行; >20词急剧退化)	加 Attention 后长短句也能处理
Bi-LSTM 分类模型三层结构	Embedding → BiLSTM → CTC Loss	Sequential([Embedding, BiLSTM, CTC Loss])

	$\text{Dense}(\text{sigmoid}, y)$ Dense(sigmoid), 这是用于分类的标准三层	$\text{Dense}(\text{ReLU}, \text{LSTM})$ Dense(ReLU)
W6 考试陷阱 (Exam Traps)		
Trap (陷阱) Bi-LSTM 可以做文本生成?	Correct Answer (正确答案) ✗ 不能! 反向LSTM需要未来上下文(整个序列预先存在) → 无法自回归生成; 只能做分	Example (示例) 生成必须用单向LSTM

Teacher Forcing 在推理时也用?	类/NER/QA ✗ 推理时用回旧(模型自己的预测)! Teacher Forcing 只在训练时用(输入真实的前一个词)	训练=真实输入+推理=自身预测
Seq2Seq 没有信息丢失?	✗ 有! 固定长度向量瓶颈 → 长序列严重丢失信息。Attention 才解决这个问题	100词 → 256词向量 信息被压缩丢失
Attention 让模型可以并行吗?	✗ RNN+Attention 仍然是顺序处理!	RNN 必须逐步处理

Term (术语)	Definition (定义)	Example (示例)
Student Model (学生模型)	蒸馏中用于学习的模型，从教师的软标签中学习	DistilBERT (66M params)
Pruning (剪枝)	移除不重要的权重/神经元/层，基于幅度(Magnitude-Based)或重要性(Importance-Based)剪枝	移除90%的小权重 → 需重训来恢复准确率
Structured vs Unstructured Pruning	结构化: 移除整个神经元/层; 非结构化: 移除权重	结构化硬件友好, 非结构化压缩率更高

Transfer Learning & PEFT (迁移学习与参数高效微调)

Term (术语)	Definition (定义)	Example (示例)
Transfer Learning (迁移学习)	高维预训练模型，只训练新添加的分支头 (0.1%参数)，最快最省	高维BERT → 加Dense层 → 训练分类
Full Fine-tuning (全量微调)	调整模型的全部参数 (100%)，效果最好	7B模型全量微调需要大量GPU内存
SFT (Supervised Fine-tuning, 监督微调)	在指令-响应对上做监督微调，GPT-3用1.3万条，LLaMA 3用1000万条	高质量标注数据的数据量决定微调效果
PEFT (Parameter-Efficient Fine-Tuning)	只调少量参数的微调方法总称: Prompt/Prefix, Adapters, BiFFit, LoRA	不调所有参数，只调一部分!
LoRA (Low-Rank Adaptation)	冻结预训练，只训练两个低秩矩阵 $A(d \times r)$ 和 $B(r \times d)$, $r \ll d=16$, 参数减少96%	$W = W_0 \Delta W$, $\Delta W = AB$, $d = 2dr$ ($r \ll d$)
LoRA 应用位置	通常应用于 Transformer 的注意力权重矩阵 W_q, W_k, W_v, W_o	只在注意力层加适配器
LoRA 实证发现	LoRA 需要比全量微调稍高的学习率; 在大 batch size 下收敛更快	LoRA lr > Full FT lr
QLoRA	LoRA + 4-bit 量化基座模型; 可在消费级 GPU 上微调大模型!	65B模型 → 48GB GPU 可训练!

Prompt Engineering (提示工程)

Term (术语)	Definition (定义)	Example (示例)
Prompt Engineering (提示工程)	向 LLM 提出正确指令以获得最佳输出; 输出格式: 仅用自然语言提示即可交互	提示任务类型: 控制输出/减轻偏见/解释性
Prompt Elements (提示元素)	一个完整 prompt 由 4 部分组成: Context (上下文) + Instruction (指令) + Input (输入) + Output Indicator (输出指示)	'你是数据科学家(Context), 做两部分分析(Instruction), 文本...(Input), Sentiment(Output)'
Zero-shot (零样本提示)	只给任务指令，不给任何示例，依靠模型自身知识	"Translate: Hello → French" → "Bonjour"
One-shot (单样本提示)	任务指令 + 1个示例，给模型格式参考	"great" → Positive. Classify. "I love this movie!"
Few-shot (少样本提示)	任务指令 + 2-5个示例，适合复杂或歧义任务	GPT-3 泛化 "Language Models are Few-Shot Learners"
Chain-of-Thought (思维链, CoT)	"Let's think step by step" → 强制模型展示推理过程，适合数学/逻辑/多步推理	"5-2=? Step1: 从5开始 Step2: 减2 Step3: 答案3"
Chain Prompts (链式提示)	多个 prompt 串联处理复杂任务，前一 prompt 输出作为后一 prompt 的输入	步骤1: 提取关键词 → 步骤2: 根据关键词分类
Role-based Prompting (角色提示)	"You are a [domain expert]. ..." → 赋予模型特定角色身份以获得领域专业回答	"You are a hiring manager. Conduct a mock interview."
Persona-Guided Prompting (人格引导)	赋予模型特定历史人物/名人/角色的写作风格和性格	"You are Shakespeare. Write a poem about night."
System Prompt (系统提示)	设定模型的人格、约束和行为规范	"你是一个礼貌的助手，不回答政治问题"

Prompt Best Practices (提示最佳实践)

Term (术语)	Definition (定义)	Example (示例)
10 Best Practices (10条最佳实践)	1. 具体清晰 2. 提供上下文 3. 分解任务 4. 用CoT推理 5. 尝试few-shot 6. 用角色提示 7. 要求澄清 8. 注意偏见 9. 要求结构化输出 10. 迭代优化	持续调整 prompt 是关键

二. Comparisons (对比)

Quantization vs Pruning vs Distillation (压缩三法对比)

Dimension (维度)	Quantization (量化)	Pruning (剪枝)	Distillation (蒸馏)	Example (示例)
What it does (做什么)	降低数据精度 (FP32 → INT8)	移除不重要的连接/权重	训练小模型模仿大模型	精度 ↓ vs 连接 ↓ vs 全新模型
Parameters changed (参数变化)	不改变参数值，只改变精度	部分参数置零，其余不变	所有参数都改变 (新模型)	量化/蒸馏结构改变; 蒸馏训练
Re-training (重训)	✗ 训练后直接压	✗ 可能需重训	✗ 需要完全重训	量化最快; 蒸馏最快; 蒸馏需重训
Size reduction (体积缩小)	~2-8x	~2-10x	取决于学生模型设计	INT8=4x; INT4=8x
Quality loss (质量损失)	FP16级小; INT4中等	可能显著 (蒸馏)	取决于学生模型大小 (蒸馏)	BERT FP16=99.5%; INT4=95%
Example (示例)	BERT FP32 → INT8: 1.3G → 340MB	移除90%权重 → 需重训	BERT → DistilBERT: 110M → 66M	各方法优缺点对比

Transfer Learning vs Full Fine-tuning vs LoRA

Dimension (维度)	Transfer Learning (迁移学习)	Full Fine-tuning (全量微调)	LoRA (低秩适配)	Example (示例)
What changes (改什么)	冻结基座 + 训练分类头	所有参数	小适配矩阵 (A, B)	调的参数量不同
Parameters trained (训练参数)	~0.1%	100%	~0.1-1%	LoRA 极省内存
Memory (内存)	低	非常高	低	7B全量 → 40GB+; LoRA → 8GB
Speed (速度)	快	慢	中等	全量微调需数天
Catastrophic Forgetting (灾难遗忘)	低风险	高风险	低风险	全量微调可能丢失预训练知识

Distilled Models (蒸馏模型案例)

Teacher (教师)	Student (学生)	Size Reduction (缩小)	Speed (加速)	Quality (性能保留)	Example (示例)
BERT	DistilBERT	60%	2x	97%	实时/移动应用
GPT-2	DistilGPT-2	60%	2x	97%	文本生成/聊天
T5	Distilled T5	60%	>1x	96%	翻译/摘要/QA

Prompt Engineering Types (提示工程类型对比)

Type (类型)	When to use (适用场景)	Pros (优点)	Example (示例)
Zero-shot (零样本)	简单任务，模型有知识	最快，无需准备	"Translate: Hello → French"
One-shot (单样本)	需要格式引导	一个例子就能修	"great! → Pos. Classify. 'okay'"
Few-shot (少样本)	复杂/歧义任务	效果最稳定	2-5个标注示例 + 新输入
CoT (思维链)	数学/推理/多步骤	逻辑更清晰	"Let's think step by step..."
Role-based (角色)	需要领域专家回答	领域匹配度高	"You are a senior Python developer"
Persona (人格)	需要特定风格	创意输出	"You are Shakespeare"

LLM Model Sizes (模型大小对比)

Model (模型)	Params (参数量)	FP32	FP16	INT8	INT4	Example (示例)
BERT-large	340M	1.3 GB	680 MB	340 MB	170 MB	理解任务基线
LLaMA-7B	7B	28 GB	14 GB	7 GB	3.5 GB	消费级GPU可运行
LLaMA-70B	70B	280 GB	140 GB	70 GB	35 GB	需多GPU
Rule (规则)	N params	N×4	N×2	N×1	N×0.5	考试必记公式

三. Formulas (公式)

Quantization Size Calculation (量化大小计算)

Formula (公式)	Description (说明)	Example (示例)
$FP32: M_{params} \times 4 \text{ bytes}$	全精度: 每个参数占4字节	$340M \times 4 = 1.36 \text{ GB}$
$FP16: M_{params} \times 2 \text{ bytes}$	半精度: 每个参数占2字节	$340M \times 2 = 680 \text{ MB}$
$INT8: M_{params} \times 1 \text{ byte}$	8位整数: 每个参数占1字节	$340M \times 1 = 340 \text{ MB}$
$INT4: M_{params} \times 0.5 \text{ bytes}$	4位整数: 每个参数占0.5字节	$340M \times 0.5 = 170 \text{ MB}$

Floating Point Format (浮点格式)

Format (格式)	Sign (符号位)	Exponent (指数位数)	Mantissa (尾数)	Example (示例)
FP16	1	5	10	半精度: 16位 = $1 + 5 + 10$
FP32	1	8	23	单精度: 32位 = $1 + 8 + 23$
FP64	1	11	52	双精度: 64位 = $1 + 11 + 52$

Distillation Loss (蒸馏损失)

Formula (公式)	Description (说明)	Example (示例)
$\mathcal{L} = \alpha \cdot \mathcal{L}_{hard} + (1-\alpha) \cdot \mathcal{L}_{soft}$	组合损失: hard label + soft label	α 平衡两项损失
$\mathcal{L}_{hard} = \text{CrossEntropy}(y_{pred}, y_{true})$	标准交叉熵: 学生预测 vs 真实标签	监督学习部分
$\mathcal{L}_{soft} = \text{KL}(p_{student}, p_{teacher})$	KL散度: 学生 vs 教师的软概率分布	T=温度, 越大分布越软

LoRA Math (LoRA 数学)

Formula (公式)	Description (说明)	Example (示例)
$W' = W + \Delta W$	新权重 = 原始权重 + 适配器	训练后直接加入，无新增参
$\Delta W = A \times B, A \in \mathbb{R}^{d \times r}, B \in \mathbb{R}^{r \times d}$	低秩分解: d个参数 → 2dr个参数 (r < d)	r=4, d=768 → 98%参数减少
$d^2 \rightarrow 2dr$	参数量从 d^2 降到 $2dr$ (r 越小越节省)	d=768, r=4: 590K → 6K 层

四. Practical / Lab (实战结论)

Key Compression Distinctions (关键压缩区别)

Distinction (区别)	Detail (详情)	Example (示例)
量化 = 无重训	训练后直接 FP32 → INT8, ~4x 更小	BERT 1.3GB/FP32 → 340MB/INT8
剪枝可能需重训	移除连接后可能降低质量，需 fine-tune 恢复	移除90%权重 → 需训练恢复准确率
蒸馏 = 完整训练周期	从头训练学生模型模仿教师	BERT → DistilBERT: 保留97%质量
LoRA r=4-16	越小的 r = 越少参数; r太小一次拟合	r=4: 6K params/层; r=16: 24K/层
QLoRA = LoRA + 4-bit 量化基座	在消费级 GPU 上微调大模型! 量化/剪枝/蒸馏可以组合	65B模型 → 48GB GPU 可训练
		无重训得小模型 → 再量化一再剪枝

Prompt Engineering Exam Templates (提示工程考试模板)

Type (类型)	Template (模板)	When to use (适用场景)	Example (示例)
Zero-shot	"Classify: {text}"	简单任务，模型有知识	"Classify sentiment: 'I love this' → Positive"
Few-shot	"Ex: 'Greet' → 'Pos ... Classify: {text}'"	需要格式/模式引导	给2-5个示例后 classify
CoT	"Let's think step by step..."	数学、推理、多步骤	"17*247 Step1: 17*20=340 Step2: 17*4=68 Answer: 408"
Role-based	"You are a [domain] expert..."	需要领域专家回答	"You are a cardiologist. Explain chest pain."
Persona	"You are [historical figure]"	需要特定风格	"You are Shakespeare. Write a poem about night."

W13 考试陷阱 (Exam Traps)

Trap (陷阱)	Correct Answer (正确答案)	Example (示例)
FP16 vs INT8 质量损失?	FP16 = 很小 (99.5%); INT8 = 轻微; INT4 = 中等 (~95%)	BERT FP16: 99.5% of FP32
DistilBERT = 蒸馏	97% BERT性能, 40% 更小内存, 60% 更快	110M → 66M params; 推理 ~2x
LoRA 训练所有参数?	✗ 只训练小适配矩阵 (~0.1-1%) 基座模型冻结!	7B模型: 全量=7B params; LoRA=7M params
大小规格: N params × 7 bytes	FP32=4, FP16=2, INT8=1, INT4=0.5 → 必记!	$340M \times 4 = 1.36 \text{ GB}$ (FP32); $1 \times 1 = 340 \text{ MB}$ (INT8)
QLoRA 只是 LoRA?	✗ QLoRA = LoRA + 4-bit 量化基座! 量化减少内存+LoRA减少训练参数	65B模型 → 48GB GPU 可训练
Quantization 改变参数量?	✗ 量化不改变参数量, 只降低精度表示; 剪枝才移除参数	量化: 值不变精度降; 剪枝: 值=0删除
Zero-shot 总是比 Few-shot 好?	✗ Few-shot 通常更稳定! Zero-shot 只在简单任务上够用	复杂歧义任务 → Few-shot 必须
CoT 适用于所有任务?	✗ CoT 适合推理/数学/多步骤; 简单分类用 Zero/Few-shot 更高效	情感分类不需要 step-by-step

